Curso de Análise e Desenvolvimento de Sistemas

Projeto para o módulo de Desenvolvimento de Aplicativos Mobile

GOLDOFFERS – MONITORAMENTO E AGREGADOR DE OFERTAS DE JOGOS

Alunos: Guilherme de Souza Chuisso e Rogério Dorcelino

Professor Orientador: Charles

Unidade - Turno

Campo Grande / Manhã

Rio de Janeiro, RJ

https://github.com/Chuisso0/Projeto-periodo-iv.git

Sumário

Sumário

1. Introdução
1.1 Objetivo
2. Metodologia
2.1 Prototipagem e Evolução de Design
2.2 Levantamento de Requisitos
2.3 Ferramentas Utilizadas
3. Desenvolvimento
3.1 Estrutura da Interface do Usuário (UI/UX)
3.2 Estrutura de Autenticação e Persistência
3.3 Estrutura de Armazenamento de Dados (NoSQL)
3.4 Consumo de APIs Externas
4. Recursos Nativos
4.1 Sistema de Arquivos (Filesystem)
4.2 Notificações Locais (Local Notifications)
5. Considerações Finais
6. Referências
7. Anexos
7.1 Anexo A: Cronograma de Entregas
7.2 Anexo B: Estrutura da Coleção no Firestore

1. Introdução

O **GoldOffers** é um aplicativo mobile híbrido desenvolvido para auxiliar jogadores na busca pelas melhores ofertas de jogos para PC. O software integra um front-end moderno e responsivo, focado na experiência do usuário (UX), com um back-end baseado em serviços de nuvem e APIs externas para garantir dados dinâmicos e atualizados.

1.1 Objetivo

O objetivo deste projeto é solucionar a fragmentação de preços no mercado de jogos digitais. Com diversas lojas operando simultaneamente, o usuário enfrenta o desafio de comparar preços manualmente. O **GoldOffers** automatiza esse processo, oferecendo uma interface centralizada que agrega informações de catálogo e valores em tempo real, gerando economia financeira e de tempo para os usuários.

2. METODOLOGIA

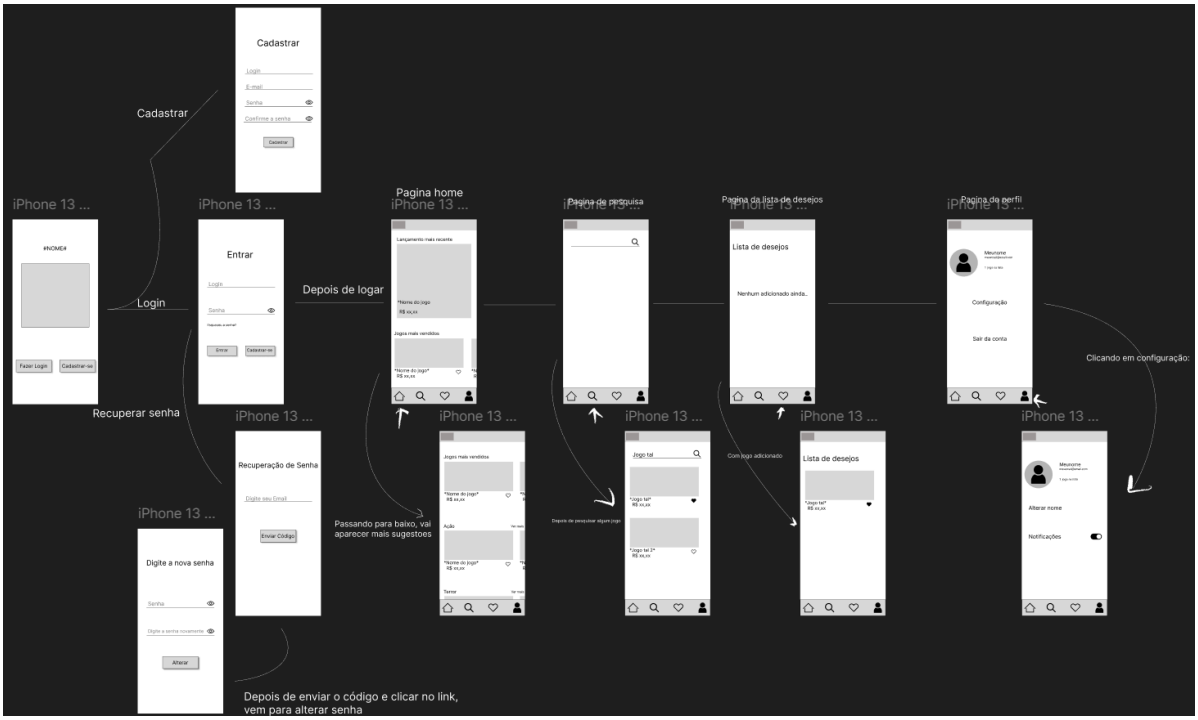
A metodologia adotada foi o desenvolvimento ágil, com foco em:

1. **Benchmarking:** Análise de soluções equivalentes no mercado.
2. **Prototipagem:** Criação de wireframes no Figma para validar o fluxo de navegação.
3. **Desenvolvimento Incremental:** Implementação das funcionalidades em Sprints, começando pelo consumo de APIs e finalizando com a persistência em nuvem e recursos nativos.

2.1. Prototipagem e Evolução de Design

https://www.figma.com/design/MlKQbohNyiEDDJZuUhtcWt/Wireframes?node-id=0-1&t=Q5ha86GRy6PuJqkO-0

O protótipo inicial foi desenvolvido no Figma para estabelecer a arquitetura de informação básica e o fluxo de navegação por abas. Durante a fase de implementação técnica, o design passou por um processo de **refinamento**, onde foram feitos ajustes para melhor adaptação aos componentes nativos do Ionic e para otimizar a visualização das capas de jogos vindas da API RAWG. Essa flexibilidade permitiu que o GoldOffers entregasse uma interface final mais robusta e alinhada às diretrizes de usabilidade mobile do que o rascunho inicial.



Protótipo no Figma

2.2. Levantamento de Requisitos

Requisitos Funcionais (RF):

- **RF01:** O sistema deve exibir um feed de jogos populares e lançamentos.
- **RF02:** O usuário deve ser capaz de pesquisar jogos por nome.
- **RF03:** O sistema deve permitir o cadastro e login de usuários.
- **RF04:** O sistema deve permitir salvar jogos em uma lista de desejos vinculada à conta.
- **RF05:** O sistema deve exportar a lista de favoritos em formato de arquivo local (.txt).

Requisitos Não Funcionais (RNF):

- **RNF01:** O app deve ser desenvolvido utilizando o framework Ionic.
- **RNF02:** A persistência de dados deve ser feita via Firebase Firestore.
- **RNF03:** O consumo de dados deve ser feito através de APIs REST gratuitas.

2.3. FERRAMENTAS

Design: Figma (Prototipagem de alta fidelidade).

Desenvolvimento: Visual Studio Code, Node.js e Angular.

Framework Mobile: Ionic Framework com Capacitor.

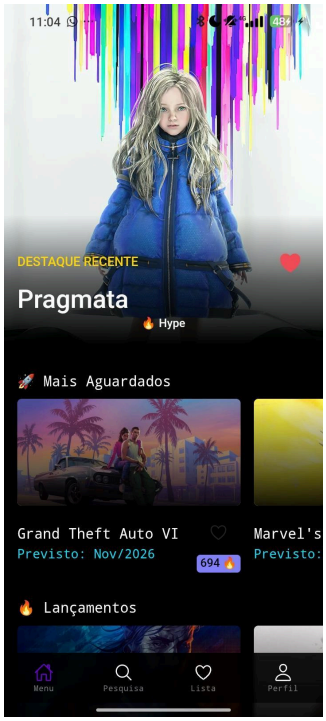
Backend & Auth: Firebase (Authentication e Cloud Firestore).

Build de Produção: Android Studio (SDK Android) para geração do APK.

3.1. ESTRUTURA DA INTERFACE DO USUÁRIO

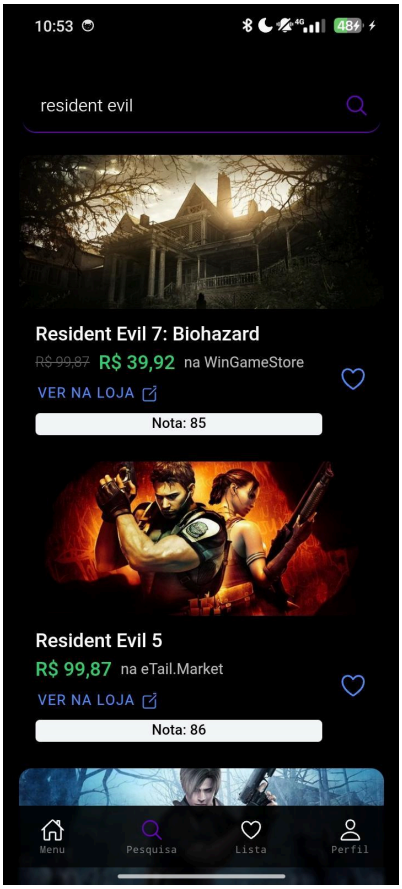
O layout foi estruturado utilizando o conceito de **Tabs (Abas)**, padrão nativo do Ionic que facilita a navegação com o polegar (User Experience - UX). A interface foi desenhada com uma estética **Noir/High-Contrast**, visando modernidade e conforto visual.

1. **Página Inicial (Tab1):** Exibe um feed dinâmico alimentado pela **API RAWG**. Utiliza componentes de *scroll* horizontal para segmentar os jogos por categorias (Hype, Lançamentos e Gêneros). Foi implementada uma estratégia de **Lazy Loading** e **Image Resizing** para garantir que a interface permaneça fluida mesmo com alto volume de mídias.



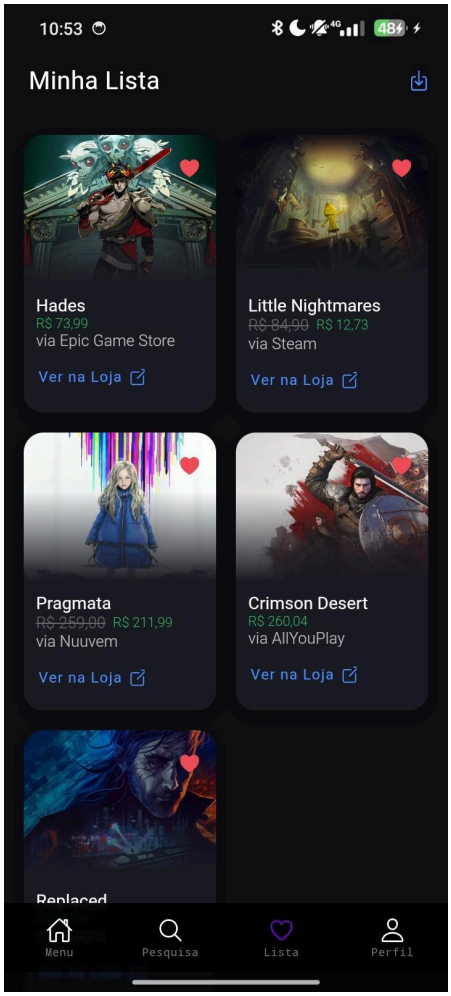
Página Inicial

2. **Busca (Tab2):** Interface de pesquisa que realiza o consumo simultâneo de duas APIs REST (**RAWG** para metadados e **IsThereAnyDeal** para cotações). O sistema filtra os resultados em tempo real, priorizando plataformas de PC e convertendo valores monetários para a moeda local (BRL).



Tela de Pesquisa

3. **Lista de Desejos (Tab3):** Área de gestão personalizada conectada ao **Cloud Firestore (Firebase)**. Esta tela utiliza o padrão **Observable** do RxJS para sincronizar os dados em tempo real. Além da visualização, a aba contém o acionamento do **recurso nativo de sistema de arquivos**, permitindo ao usuário exportar sua lista de desejos.



Tela de Lista de Desejos

3.2 ESTRUTURA DE AUTENTICAÇÃO

O aplicativo conta com uma camada de segurança baseada no **Firebase Auth**, permitindo que cada usuário tenha sua própria base de dados persistente. A interface de perfil (Tab4) inclui um **Modal customizado** para identificação dos desenvolvedores e informações acadêmicas, cumprindo os requisitos de autoria do projeto.

3.3 ESTRUTURA DE ARMAZENAMENTO DE DADOS

Nuvem (Firebase Firestore): Os dados dos jogos favoritos são salvos de forma estruturada: `usuarios/{uid}/favoritos/{id_do_jogo}`. Isso garante que o usuário acesse sua lista de qualquer dispositivo.

Arquivos (FileSystem): Implementada a exportação da lista de desejos para um arquivo **.txt** local, permitindo ao usuário ter um registro estático das ofertas.

3. 4. CONSUMO DE API

O projeto utiliza duas APIs complementares:

1. **RAWG API:** Fornece o catálogo de jogos, incluindo imagens (thumbs), metadados e notas do Metacritic. Utiliza sistema de chave de API e suporte nativo a paginação.
2. **IsThereAnyDeal (ITAD) API (V3):** Utilizada exclusivamente para a busca de preços em lojas, permitindo a comparação em tempo real.

4. Recursos nativos

Para cumprir o desafio técnico, foi utilizado o recurso nativo de **Sistema de Arquivos (Filesystem)**.

4.1. Sistema de Arquivos (Filesystem)

- **Plugin:** `@capacitor/filesystem`
- **Comando de Instalação:** `npm install @capacitor/filesystem`
- **Método Utilizado:** `Filesystem.writeFile`
- **Finalidade:** Exportar a lista de jogos favoritados pelo usuário para a pasta "Documentos" do Android. Isso permite que o usuário tenha um registro estático (.txt) das ofertas para consulta offline.

4.2. Notificações Locais (Local Notifications)

- **Plugin:** `@capacitor/local-notifications`
- **Comando de Instalação:** `npm install @capacitor/local-notifications`
- **Método Utilizado:** `LocalNotifications.schedule`
- **Finalidade:** Implementação de alertas de sistema para confirmar o sucesso do login (persistência de sessão) e notificar o usuário sobre atualizações de preços na lista de desejos. O recurso garante que o GoldOffers se comunique com o usuário mesmo quando o app não está em primeiro plano, aumentando o engajamento.

5. Considerações Finais

O projeto **GoldOffers** atingiu os objetivos propostos, integrando com sucesso o framework Ionic com serviços de nuvem do Firebase e APIs externas. Como melhorias futuras, sugere-se a implementação de notificações push (Web Push) para alertar o usuário quando um jogo da sua lista atingir um preço histórico mínimo.

6. Referências

- ANGULAR. Framework web. Disponível em: https://angular.io/. Acesso em: 28 abr. 2026.
- CAPACITOR. Filesystem Plugin API. Disponível em: https://capacitorjs.com/docs/apis/filesystem. Acesso em: 28 abr. 2026.
- FIREBASE. Cloud Firestore e Authentication. Disponível em: https://firebase.google.com/. Acesso em: 28 abr. 2026.
- IONIC FRAMEWORK. Documentação oficial para desenvolvimento mobile. Disponível em: https://ionicframework.com/docs. Acesso em: 28 abr. 2026.
- IS THERE ANY DEAL (ITAD). API v3 para preços de jogos. Disponível em: https://isthereanydeal.com/. Acesso em: 28 abr. 2026.
- RAWG API. The Biggest Video Game Database. Disponível em: https://rawg.io/apidocs. Acesso em: 28 abr. 2026.

7. Anexos

7.1. Anexo A: Cronograma de entregas

ID	Item (Atividade)	Responsável	Início	Término	Status
01	Configuração inicial do	Guilherme	02/03/2026	05/03/2026	Concluído

	projeto Ionic + GitHub				
02	Criação da tela de Login e Cadastro (UI/UX)	Guilherme/Rogério	06/03/2026	15/03/2026	Concluído
03	Integração com Firebase (Auth e Firestore)	Guilherme	16/03/2026	25/03/2026	Concluído
04	Navegação entre telas (Tabs) e Consumo de API	Rogério	26/03/2026	10/04/2026	Concluído
05	CRUD da GoldOffers (Salvar/Remover Favoritos)	Guilherme/Rogério	11/04/2026	22/04/2026	Concluído
06	Implementação de Recurso	Guilherme	23/04/2026	28/04/2026	Concluído

	Nativo (Filesystem)				
07	**Testes de Usabilidade e Refatoração de Código**	Guilherme/Rogério	29/04/2026	02/05/2026	Pendente
08	**Geração do APK Final e Entrega da Documentação**	Guilherme/Rogério	03/05/2026	05/05/2026	Pendente

7.2. Anexo B: Exemplo - Comando de criação das tabelas

Como o projeto utiliza o **Cloud Firestore (Banco de Dados NoSQL)**, a criação de tabelas é dinâmica através de documentos e coleções. Abaixo, a representação da estrutura de dados utilizada para salvar os favoritos dos usuários:

Coleção Principal: `usuarios`

Subcoleção: `favoritos`

{
  "id": "46253",
  "itadId": "resident-evil-4",
  "nome": "Resident Evil 4",
  "thumb": "https://media.rawg.io/media/games/resident-evil-4-remake.jpg",
  "adicionadoEm": "2026-04-28T13:20:00.000Z",
  "emailUsuario": "gui_chuisso1@outlook.com"
}